Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Proyecto del curso



ER-AFD

Convertidor de expresiones regulares a autómatas finitos deterministas

Documento 02

Manual de usuario

Integrantes

Carlos Abraham Méndez Asencio - Carné 94902417370

Maycol Alexander Ortiz Flores - Carné 9490-24-27213

Preparación y ejecución del programa

ER-AFD es una herramienta de línea de comandos desarrollada en Python para Linux de 64 bits. Se requieren Python 3.10 o posterior y GNU Make. La construcción se realiza con bibliotecas estándar, sin instalar paquetes con pip ni utilizar permisos de administrador.

En la máquina virtual utilizada para la verificación, el proyecto está ubicado en el directorio ~/er-afd. Desde una terminal puede ejecutarse lo siguiente:

```
cd ~/er-afd
make config
make build
export PATH="$PWD/dist:$PATH"
```

make config revisa el entorno y make build prepara los ejecutables dist/er-afd y dist/er-lex. La modificación del PATH es temporal y solo afecta a la terminal actual. Si se prefiere, pueden ejecutarse directamente los archivos de la carpeta dist.

Reconocer una cadena

```
er-afd '(a|b)*abb' 'aabb'
estado=$?
printf 'Código: %s\n' "$estado"
```

El programa presenta el AFND, el AFD, sus estados iniciales y finales, alfabetos, transiciones y el recorrido realizado. En este caso, aabb se acepta con el código 1. Ese valor se utiliza por exigencia del proyecto aunque los intérpretes Unix normalmente consideran distinto de cero como un estado no exitoso.

Código de salida	Interpretación
1	Cadena aceptada.
0	Cadena rechazada: la evaluación terminó, pero no en un estado final.
2	Error de entrada, sintaxis o ejecución.

Si se utiliza set -e en Bash, el código de aceptación 1 debe capturarse mediante una condición apropiada. No conviene encadenar el programa con && esperando que la aceptación se represente con cero.

El entorno documentado para las pruebas fue Ubuntu 26.04.1 LTS x86_64, Python 3.14.4 y GNU Make 4.4.1, con el usuario regular erafd.

Expresiones regulares, espacios y escapes

El comando recibe una expresión regular y la cadena que se desea evaluar. Las operaciones aceptadas se resumen en la siguiente tabla.

Operación	ER	Ejemplos aceptados
Concatenación	ab	ab
Unión	a|b	a o b

Operación	ER	Ejemplos aceptados
Cero o más	a*	Vacío, a, aa, ...
Agrupación	(ab)*	Vacío, ab, abab, ...
Cualquier ASCII	.	Un carácter, incluido LF
Cadena vacía	ε	Vacío
Conjunto o rango	[abc], [a-z]	Un símbolo del conjunto
Complemento	[^abc]	Un ASCII diferente de a, b o c
Escapes especiales	\N, \A, \a, \z	Dígitos, mayúsculas, minúsculas o letras
Códigos hexadecimales	\h20, \h41	Espacio o A

La precedencia se aplica en este orden: agrupación, cerradura *, concatenación y unión. No existe un símbolo especial de concatenación: escribir ab significa leer a y después b. Para representar una o más repeticiones se puede escribir rr*.

Uso de comillas

```
er-afd 'hola mundo' 'hola mundo'
er-afd 'hola\ mundo' 'hola mundo'
er-afd 'hola\h20mundo' 'hola mundo'
er-afd 'a\*' 'a*'
er-afd 'ε' ''
```

Las comillas simples permiten que Bash entregue la expresión al programa sin interpretar las barras invertidas. Las comillas dobles también agrupan argumentos, pero tienen reglas distintas para algunos escapes. En el lenguaje de ER-AFD, \n y \t significan las letras n y t; el salto de línea y el tabulador se indican con \h0A y \h09.

El texto de entrada se compara exactamente como fue recibido: el programa no elimina automáticamente espacios ni saltos de línea. Cuando una rama debe representar la cadena vacía se escribe ε.

Validación de archivos y exportaciones

Leer una entrada desde un archivo

```
printf 'aabb' > cadena.txt
er-afd '(a|b)*abb' --file cadena.txt
er-afd '(a|b)*abb\h0A' --file examples/cadena.txt
```

El archivo se lee como bytes ASCII. Si termina con un salto de línea, ese carácter forma parte de la cadena y también debe estar contemplado por la expresión regular. El parámetro --file y una cadena como argumento son alternativas excluyentes.

Exportar información del autómata

```
er-afd 'a*b' 'aaab' --json
er-afd 'a*b' 'aaab' --dot automata
er-afd --help
```

La opción --json presenta estados, alfabetos, subconjuntos, transiciones, entrada, recorrido y exit_code. La opción --dot escribe los archivos automata.afnd.dot y automata.afd.dot; si Graphviz está instalado, se pueden convertir posteriormente a imágenes.

```
dot -Tpng automata.afd.dot -o afd.png
```

Interpretar las tablas

```
q0|a|{q1,q2}    # AFND: puede haber varios destinos
Q0|a|Q1        # AFD: existe un solo destino
```

Los estados iniciales se identifican como q0 y Q0, y F enumera los finales. Σ conserva las etiquetas originales de la expresión, mientras que Γ representa las clases de caracteres disjuntas utilizadas para completar la matriz. Cuando una entrada no permite continuar, la transición puede conducir al estado pozo, que no es final.

Límites del programa

```
er-afd '(a|b)*abb' 'aabb' --max-states 2048
```

El parámetro --max-states cambia el máximo de estados deterministas permitido. Un valor mayor puede requerir más memoria. Cuando se supera el límite, el programa devuelve un error 2 en lugar de registrar una cadena rechazada.

Tokenización con varias reglas

er-lex permite reconocer tokens en textos o archivos fuente mediante varias expresiones regulares. Las reglas se declaran en una lista JSON y se evalúan en su orden original.

```
python3 dist/er-lex --rules examples/reglas.json \
  --file examples/programa.txt
```

Ejemplo de reglas

```
[
  {"name":"ESPACIO", "regex":"\\h20\\h20*", "skip":true},
  {"name":"SI", "regex":"if"},
  {"name":"ID", "regex":"\\z(\\z|\\N)*"},
  {"name":"NUMERO", "regex":"\\N\\N*"}
]
```

El tokenizador selecciona la coincidencia más larga y, cuando dos reglas tienen la misma longitud, utiliza la que aparece primero en el JSON. De esta forma, if puede ser una palabra reservada mientras ifx se reconoce como identificador. Las reglas skip consumen caracteres sin mostrar un token en la salida.

Campo del token	Significado
type / value	Tipo reconocido y texto asociado.
start / end	Desplazamientos desde cero; end es exclusivo.
line / column	Línea y columna desde uno; LF inicia una nueva línea.

El ejemplo de entrada if contador = 12 + valor; produce los tipos SI, IDENTIFICADOR, IGUAL, NUMERO, SUMA, IDENTIFICADOR y PUNTO_COMA. Los espacios se omiten. No pueden utilizarse reglas que acepten una cadena vacía, ya que impedirían avanzar de forma segura.

Pruebas y solución de problemas

Antes de entregar el proyecto es recomendable repetir las verificaciones desde la carpeta principal:

```
bash tools/verify_linux.sh
make test
make run ER="(a|b)*abb" CADENA="aabb"
make clean
make build
```

El verificador prepara una carpeta dentro de doc/evidencias/linux/ y registra el resultado de cada comprobación. Si el proceso concluye correctamente, se muestra VERIFICACION LINUX COMPLETADA y resultado.json debe contener completed: true. El comando make clean conserva las fuentes, la documentación y las evidencias.

Problema	Revisión recomendada
La cadena se rechaza inesperadamente	Comprobar si la ER debe reconocer la cadena completa y si el archivo contiene espacios o LF final.
No se reconoce er-afd	Ejecutar make build y agregar dist al PATH, o invocar python3 dist/er-afd.
Error de sintaxis	Revisar la posición indicada, los paréntesis, corchetes y rangos.
Carácter no ASCII	Usar los códigos 00..7F; los caracteres acentuados no pertenecen a ASCII.
Argumento que comienza con -	Colocar el separador -- antes de argumentos posicionales.
Retorno 1 en Bash	Recordar que significa aceptación para este proyecto y capturarlo correctamente.
Falta Python o Make	Instalar las herramientas necesarias antes de iniciar la construcción.

Ejecución directa

```
python3 -m er_afd 'a*b' 'aaab'
python3 tools/build.py config
python3 tools/build.py build
```

Ubuntu registró 39 pruebas, 1,499 casos y 20,306 comparaciones; el procedimiento para repetirlas está en PASOS_PRUEBAS_UBUNTU.md.